

~\OneDrive\Desktop\artificial neural networks\lab3\lab3_01.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # =====
5 # PARAMETERS
6 # =====
7 N = 3
8 n_reservoir = 500
9 k_ridge = 0.01
10
11 var_w_in = 0.002
12 var_w = 2.0/500.0
13
14 # =====
15 # LOAD DATA
16 # =====
17 print("Loading data...")
18 train_data = np.loadtxt('training-set.csv', delimiter=',')
19 test_data = np.loadtxt('test-set-10.csv', delimiter=',')
20
21 if train_data.shape[0] == 3:
22     train_data = train_data.T
23 if test_data.shape[0] == 3:
24     test_data = test_data.T
25
26 print(f"Training shape: {train_data.shape}")
27 print(f"Test shape: {test_data.shape}")
28
29 T_train = train_data.shape[0]
30 T_test = test_data.shape[0]
31
32 # =====
33 # INITIALIZE WEIGHTS
34 # =====
35 print("\nInitializing weights...")
36 np.random.seed(42)
37
38 W_in = np.random.randn(n_reservoir, N) * np.sqrt(var_w_in)
39 W = np.random.randn(n_reservoir, n_reservoir) * np.sqrt(var_w)
40
41 # =====
42 # TRAINING PHASE
43 # =====
44 print("\n--- TRAINING ---")
45 r = np.zeros(n_reservoir)
46
47 # Store states for regression
48 R_list = []
49 Y_list = []
50
51 warmup = 100
```

```
52
53 for t in range(T_train - 1):
54     # Update reservoir
55     r = np.tanh(W @ r + W_in @ train_data[t, :])
56
57     # Store for regression (after warmup)
58     if t >= warmup:
59         R_list.append(r.copy())
60         Y_list.append(train_data[t+1, :])
61
62 print(f"Collected {len(R_list)} samples for training")
63
64 # =====
65 # TRAIN OUTPUT WEIGHTS
66 # =====
67 print("\nTraining output weights...")
68 R = np.array(R_list) # shape: (T_train-warmup-1, n_reservoir)
69 Y = np.array(Y_list) # shape: (T_train-warmup-1, N)
70
71 W_out = np.linalg.solve(R.T @ R + k_ridge * np.eye(n_reservoir), R.T @ Y)
72
73 train_error = np.mean((R @ W_out - Y)**2)
74 print(f"Training MSE: {train_error:.6f}")
75
76 # =====
77 # TEST PHASE - Feed test data through network
78 # =====
79 print("\n--- FEEDING TEST DATA ---")
80
81 # Continue from last training state
82 for t in range(T_test):
83     r = np.tanh(W @ r + W_in @ test_data[t, :])
84
85 print("Test data processed, starting predictions from this state")
86
87 # =====
88 # AUTOREGRESSIVE PREDICTION
89 # =====
90 print("\n--- PREDICTIONS ---")
91 n_predict = 500
92 predictions = np.zeros((n_predict, N))
93
94 for t in range(n_predict):
95     # Compute output
96     o = W_out.T @ r
97     predictions[t, :] = o
98
99     # Update reservoir with output
100    r = np.tanh(W @ r + W_in @ o)
101
102    if (t+1) % 100 == 0:
103        print(f"  step {t+1}/{n_predict}")
104
105 # =====
```

```
106 # SAVE
107 # =====
108 print("\nSaving...")
109 y_predictions = predictions[:, 1]
110
111 np.savetxt('prediction.csv', y_predictions, delimiter=',', fmt='%.10f')
112 print(f"Saved {len(y_predictions)} predictions")
113
114 '''(optional), uncomment to plot
115 # =====
116 # PLOTS
117 # =====
118 fig, axes = plt.subplots(2, 2, figsize=(12, 8))
119
120 axes[0, 0].plot(train_data[:, 1], 'b-', linewidth=0.5)
121 axes[0, 0].set_title('Training y')
122 axes[0, 0].grid(True, alpha=0.3)
123
124 axes[0, 1].plot(test_data[:, 1], 'g-', linewidth=0.5)
125 axes[0, 1].set_title('Test y')
126 axes[0, 1].grid(True, alpha=0.3)
127
128 axes[1, 0].plot(y_predictions, 'r-', linewidth=0.8)
129 axes[1, 0].set_title('Predictions y')
130 axes[1, 0].grid(True, alpha=0.3)
131
132 axes[1, 1].plot(predictions[:, 0], predictions[:, 2], 'r-', linewidth=0.5)
133 axes[1, 1].set_title('Attractor')
134 axes[1, 1].set_xlabel('x')
135 axes[1, 1].set_ylabel('z')
136 axes[1, 1].grid(True, alpha=0.3)
137
138 plt.tight_layout()
139 plt.savefig('results.png', dpi=150)
140 plt.show()
141 '''
```